BASIC 25 | INTERMEDIATE 40 | SCENARIO 35

CI/CD and Azure DevOps

Interview Questions and Answers

100 questions for candidates with 5+ years of experience
Basic, intermediate and scenario-based, with YAML examples.

25
Basic questions

40
Intermediate questions

35
Scenario questions

Covers: pipelines, agents, security, IaC, containers, AKS, deployments, troubleshooting

How to use this guide

At 5+ years, interviewers are not checking whether you know what a pipeline is. They want to hear **decisions, trade-offs, and things that went wrong**. Every answer below is short enough to say aloud in about a minute. Add one real example from your own work to each, because that is what separates a memorised answer from an experienced one.

Level	Count	What the interviewer is testing	How to answer
Basic	25	Vocabulary and fundamentals. A wrong answer here is a red flag at your level.	Crisp, accurate, 30 seconds.
Intermediate	40	Hands-on depth: YAML details, security, IaC, containers, deployment strategies.	Definition, then how you used it, then a gotcha.
Scenario	35	Design thinking, troubleshooting, judgment under pressure, communication.	Use the framework below.

A framework for scenario questions

Step	What to say
1. Clarify	Ask 1-2 questions: scale, constraints, current tooling, risk tolerance.
2. Contain	For incidents: stop the bleeding first (rollback, disable, isolate), then investigate.
3. Diagnose	State how you would find the root cause: logs, recent changes, differences between environments.
4. Fix	Give the immediate fix and the long-term fix separately.
5. Prevent	Close with what you would add so it cannot happen again (check, gate, alert, test, policy).

Topics to be ready for

Area	Typical keywords the interviewer listens for
Pipelines	YAML, stages/jobs/steps, templates, extends, parameters, conditions, matrix, caching, output variables
Agents	Microsoft-hosted, self-hosted, scale sets, Managed DevOps Pools, demands, capacity
Security	Workload identity federation, Key Vault, least privilege, secret scanning, SAST/SCA, supply chain, SBOM, signing
Deployments	Environments, approvals and checks, blue-green, canary, rolling, slots, rollback, build once deploy many
IaC	Terraform state, plan/apply, Bicep, drift, modules
Containers	Multi-stage Docker, ACR, AKS, Helm, GitOps, image tagging, pull errors
Quality	Test pyramid, quality gates, flaky tests, branch policies, trunk-based development
Operations	DORA metrics, monitoring, audit, cost, governance at scale

> **Honesty rule**
> If you have not used a feature (for example Managed DevOps Pools), say so, then explain how you would approach it. Interviewers respect "I have not used it, but I would..." far more than a bluffed answer that falls apart on the follow-up question.

A Basic questions (1-25)

Fundamentals you must answer instantly

1 What is CI/CD? Explain the difference between continuous integration, continuous delivery and continuous deployment.

BASIC | CI/CD concepts

Continuous integration: developers merge small changes often, and every merge triggers an automated build and test run, so integration problems surface within minutes.

Continuous delivery: every change that passes the pipeline is always in a releasable state, but a human decides when to release (usually an approval).

Continuous deployment: goes one step further: every change that passes all automated checks is released to production automatically, with no manual approval.

2 Why do teams adopt CI/CD? What problems does it solve?

BASIC | CI/CD concepts

- **Faster feedback:** bugs are found minutes after the commit, when they are cheapest to fix.
- **Smaller, safer releases:** small changes are easier to review, test and roll back.
- **Consistency:** the same automated process runs every time, removing "works on my machine" and manual errors.
- **Speed and frequency:** teams ship more often, which improves lead time and recovery time.
- **Traceability:** every release links back to a commit, a build and an approval.

3 What is Azure DevOps and what are its main services?

BASIC | Azure DevOps

Azure DevOps is Microsoft's platform for the whole delivery lifecycle. It has five services: **Boards** (work tracking, Agile/Scrum), **Repos** (Git hosting, pull requests, branch policies), **Pipelines** (CI/CD), **Test Plans** (manual and exploratory testing) and **Artifacts** (hosted NuGet, npm, Maven, Python and universal package feeds). You can use them together or separately, for example Pipelines with code hosted on GitHub.

4 What is the difference between Classic and YAML pipelines? Which do you prefer?

BASIC | Pipelines

Classic pipelines are built in a graphical editor and stored in the service, not in the repo. YAML pipelines are code in the repository.

I prefer YAML: it is versioned with the application, reviewed in pull requests, branch-aware, reusable through templates, and easy to audit and roll back. Classic is harder to reuse and compare, and Microsoft's investment is in YAML. Classic release pipelines are mostly legacy, and the replacement is multi-stage YAML with environments.

5 Describe the structure of an Azure DevOps YAML pipeline.

BASIC | Pipelines

The hierarchy is **pipeline, stages, jobs, steps**. A stage is a major phase (Build, Deploy) and can have approvals. A job runs on one agent, and jobs in a stage run in parallel unless `dependsOn` orders them. A step is a `script` or a `task`. You can skip levels: a pipeline with only `steps` is valid, and Azure DevOps creates the stage and job implicitly. Root-level keys include `trigger`, `pr`, `pool`, `variables`, `parameters` and `resources`.

6 What is an agent and an agent pool? Microsoft-hosted vs self-hosted?

BASIC | Agents

An agent is the machine and software that runs a job. A pool is a group of agents that jobs queue against.

Microsoft-hosted: fresh clean VM per job, no maintenance, preinstalled tools, but no access to private networks and limited customisation.

Self-hosted: you run it (VM, container, scale set), so you can reach private networks, use custom tools or bigger hardware, and cache between runs. The cost is patching, security and scaling.

7 What is a service connection?

BASIC | Security

A service connection is a secured, named credential that lets a pipeline reach an external system: Azure subscriptions, container registries, Kubernetes, GitHub, SonarQube and so on. Pipelines refer to it by name and never see the secret. For Azure, the recommended type uses **workload identity federation** (OIDC), which needs no stored secret. You can restrict which pipelines may use it and add approvals and checks to it.

8 What are the variable syntaxes in Azure DevOps and when is each evaluated?

BASIC | Pipelines

- `$(var)` **macro:** replaced at runtime just before a task runs. The most common.
- `${{ variables.var }}` **template expression:** evaluated at compile time, when the YAML is expanded. Used for parameters and conditional insertion.
- `$[ variables.var ]` **runtime expression:** evaluated at runtime, used in conditions and when assigning variables from outputs.

A common bug is using a macro where a compile-time value is needed, or reading a value that does not exist yet at compile time.

9 What are variable groups and how do they connect to Azure Key Vault?

BASIC | Security

A variable group is a shared set of variables, reusable across pipelines and managed in the Library. You can mark values secret. A group can also be **linked to Key Vault**: pick the vault and the secrets, and they are fetched at runtime through a service connection. Alternatively use the `AzureKeyVault@2` task inside the pipeline. Key Vault is the better source of truth because rotation happens in one place and access is audited.

10 What types of triggers exist in Azure Pipelines?

BASIC | Pipelines

- **CI trigger** (`trigger`): runs on pushes, filtered by branches, tags and paths.
- **PR trigger** (`pr`): validates pull requests. For Azure Repos it is configured through branch policy; for GitHub it is in YAML.
- **Scheduled** (`schedules`): cron, in UTC.
- **Pipeline resource trigger:** runs when another pipeline completes.
- **Manual** and **API/CLI** runs.

Setting `trigger: none` disables CI so the pipeline is manual or scheduled only.

11 What is the difference between pipeline artifacts, build artifacts and Azure Artifacts?

BASIC | Artifacts

Pipeline artifacts (`publish` / `download`) are the current, faster mechanism for passing files between stages and jobs of a run. **Build artifacts** (`PublishBuildArtifacts`) are the older mechanism. **Azure Artifacts** is a different thing: a package feed (NuGet, npm, Maven, Python) for sharing versioned packages across teams and pipelines, with upstream sources such as nuget.org.

12 What is an Environment in Azure Pipelines?

BASIC | Deployments

An environment represents a deployment target such as dev, test or prod. It gives you deployment history per target, traceability back to the run and commit, and a place to attach **approvals and checks**. Environments can also have resources (Kubernetes namespace, virtual machines). A job that deploys uses `deployment:` with `environment:` to get these features.

13 What is the difference between a script step and a task?

BASIC | Pipelines

`script` (and `bash`, `pwsh`) runs a command in a shell. A `task` is a packaged, versioned unit of functionality (`Name@version`) with defined inputs, such as `AzureWebApp@1` or `Docker@2`. Use a script for simple commands. Use a task when it handles authentication, logging and options for you. Pin the major version of every task.

14 What are branch policies and why do they matter?

BASIC | Repos

Branch policies protect important branches such as `main`. Typical rules: require pull requests, a minimum number of reviewers, linked work items, resolved comments, and a **successful build validation** before merge. They stop direct pushes, keep `main` releasable, and provide the audit trail many compliance regimes need.

15 Compare GitFlow and trunk-based development.

BASIC | Git

GitFlow uses long-lived `develop`, `release` and `hotfix` branches: suited to versioned products with scheduled releases, but it creates merge pain and delays integration. **Trunk-based** development uses short-lived branches (or direct commits) merged into `main` at least daily, with feature flags hiding unfinished work. It fits CI/CD best because integration is constant, and DORA research links it with higher delivery performance.

16 What is Infrastructure as Code and which tools have you used?

BASIC | IaC

IaC defines infrastructure in versioned files so environments are reproducible, reviewable and automated. Declarative tools: **Terraform** (multi-cloud, state file), **Bicep/ARM** (Azure-native, no state file), Pulumi (general-purpose languages). Configuration tools: Ansible. Benefits: consistency across environments, easy rebuilds, drift detection, and changes reviewed like code.

17 What is Docker and how does it fit in CI/CD? Image vs container?

BASIC | Containers

Docker packages an app with its dependencies so it runs the same everywhere. An **image** is the immutable, layered template; a **container** is a running instance of an image. In CI/CD the pipeline builds an image once, scans and tags it, pushes it to a registry (ACR), and the same image is deployed to every environment. That removes environment differences and supports "build once, deploy many".

18 What happened to build and release pipelines? How do they map to YAML?

BASIC | Pipelines

In Classic, **build** pipelines produced artifacts and **release** pipelines deployed them through stages. Multi-stage YAML merges both: a Build stage publishes artifacts, then Deploy stages consume them, with environments providing approvals and gates. The pipeline file now describes the whole path from commit to production in one reviewable place.

19 What is a deployment job and what strategies does it support?

BASIC | Deployments

A deployment job (`deployment:`) targets an environment and records deployment history. Its `strategy` can be **runOnce** (run steps once), **rolling** (update in batches, VM targets), or **canary** (incrementally expose to more traffic). Lifecycle hooks include `preDeploy`, `deploy`, `routeTraffic`, `postRouteTraffic`, and `on: success/failure` for rollback. Deployment jobs automatically download published artifacts.

20 How does parallelism work and what limits it?

BASIC | Agents

A **parallel job** is one job running at a time on an agent. Your organisation has a number of parallel jobs for Microsoft-hosted agents and another for self-hosted. If you exceed it, jobs queue. Free tiers are limited, so for real workloads you buy more parallel jobs or add self-hosted agents. Within a pipeline, jobs run in parallel unless `dependsOn` serialises them, and `matrix` can fan out many parallel jobs.

21 How do you handle secrets in a pipeline?

BASIC | Security

Never put secrets in YAML or the repo. Store them in Key Vault or as secret variables. Secret variables are masked in logs but are **not** passed to scripts automatically: map them explicitly with `env:`. Prefer removing secrets entirely with workload identity federation or managed identity. Do not echo them, restrict who can edit pipelines, and avoid exposing secrets to fork pull requests.

secret-to-env.yml

```yaml
- script: ./deploy.sh
  env:
    DB_PASSWORD: $(dbPassword)
```

22 What do dependsOn and condition do?

BASIC | Pipelines

`dependsOn` sets execution order between stages or jobs (and builds a dependency graph, so independent jobs still run in parallel). `condition` decides whether something runs. The default is `succeeded()`. Others: `failed()`, `always()`, `succeededOrFailed()`, or an expression such as `eq(variables['Build.SourceBranch'], 'refs/heads/main')`. `always()` ignores cancellation too, so `succeededOrFailed()` is usually safer for cleanup.

23 How do you version builds and releases?

BASIC | Versioning

Use **semantic versioning** (MAJOR.MINOR.PATCH) for what you publish, with the build number or commit SHA appended for traceability. In Azure DevOps the run number format is set with `name:` (for example `1.4$(Rev:.r)`). Tag container images with a unique value such as the build ID or commit SHA, never only `latest`. Tag the Git commit of each release.

24 What is the purpose of a pull request in a CI/CD workflow?

BASIC | Repos

A PR is the quality gate before code reaches `main`. It triggers build validation (compile, tests, scans), enables peer review, links work items, and creates an auditable record. Combined with branch policies, nothing merges unless it passes automated checks and review, which keeps the main branch always deployable.

25 How does Azure DevOps compare with GitHub Actions?

BASIC | **Tools**

Both use YAML and run on hosted or self-hosted runners. Azure DevOps is an all-in-one suite (Boards, Repos, Test Plans, Artifacts) with strong enterprise controls: stages, environments with checks, variable groups and service connections. GitHub Actions is tied to the GitHub repo and has a huge marketplace, simple event model and OIDC login. Key differences: ADO has a built-in `stages` layer and auto-checkout; GitHub Actions needs `actions/checkout` and uses `needs` between jobs. Many organisations use GitHub for code and Azure Pipelines or Actions for CI/CD.

B Intermediate questions (26-65)

Hands-on depth: this is where most interviews are decided

26 What are YAML templates and why use them? What types are there?

INTERMEDIATE | Pipelines

Templates let you define reusable pipeline pieces once and consume them everywhere. Types: **step**, **job**, **stage** and **variable** templates. They accept typed `parameters`. Benefits: no copy-paste, one place to fix or upgrade, and a standard way of building across teams. Templates can live in a separate repository and be referenced through `resources: repositories`, and you pin them to a tag for stability.

templates.yml

```yaml
# templates/build.yml
parameters:
  - name: project
    type: string
steps:
  - script: dotnet build ${{ parameters.project }}

# azure-pipelines.yml
steps:
  - template: templates/build.yml
    parameters:
      project: src/Api/Api.csproj
```

27 What is the difference between runtime parameters and variables?

INTERMEDIATE | Pipelines

Parameters are typed inputs, resolved at **compile time**, and can be shown as a form when you manually run the pipeline (with allowed values). They can change the pipeline's structure with `${{ if }}` or `${{ each }}`. **Variables** are strings available at runtime (macro) and can be changed by steps. Use parameters for choices that alter what is built (environment, skip tests), and variables for values that tasks consume.

28 How do you pass a value from one step, job or stage to another?

INTERMEDIATE | Pipelines

Set an **output variable** with the logging command and `isOutput=true`, give the step a `name`, then read it through `dependencies` (job to job, same stage) or `stageDependencies` (stage to stage). Variables set without `isOutput` only live within the same job. For files, use pipeline artifacts instead.

output-variables.yml

```yaml
jobs:
  - job: A
    steps:
      - bash: echo "##vso[task.setvariable variable=ver;isOutput=true]1.4.2"
        name: setVer
  - job: B
    dependsOn: A
    variables:
      ver: $[ dependencies.A.outputs['setVer.ver'] ]
    steps:
      - script: echo $(ver)
```

29 What is a matrix strategy and where would you use it?

INTERMEDIATE | Pipelines

A `matrix` generates multiple copies of a job, each with different variable values, running in parallel. Typical uses: test across several runtime versions (.NET, Node, Java), operating systems, or database versions, and sharding a large test suite. You can cap concurrency with `maxParallel`. The result is broader coverage without making the pipeline slower.

matrix.yml

```yaml
strategy:
  matrix:
    node18:
      nodeVersion: '18.x'
    node20:
      nodeVersion: '20.x'
  maxParallel: 2
```

30 How do you use caching in Azure Pipelines?

INTERMEDIATE | Performance

The `Cache@2` task saves a folder keyed by a hash of the lock file and restores it on later runs, skipping downloads for npm, NuGet, Maven or Docker layers. Choose the key carefully (OS plus lock file hash), add `restoreKeys` for partial hits, and cache only dependency folders, never build output that should be rebuilt. A stale cache causes confusing bugs, so changing the key busts it.

cache.yml

```yaml
- task: Cache@2
  inputs:
    key: 'npm | "$(Agent.OS)" | package-lock.json'
    restoreKeys: |
      npm | "$(Agent.OS)"
    path: $(Pipeline.Workspace)/.npm
- script: npm ci --cache $(Pipeline.Workspace)/.npm
```

31 What approvals and checks can you put on an environment or resource?

INTERMEDIATE | Deployments

Approvals (named users or groups, with timeout), **branch control** (only deploy from `main`), **business hours**, **exclusive lock** (one deployment at a time), **required template** (pipeline must extend an approved template), **Invoke Azure Function**, **Invoke REST API**, **Query Azure Monitor alerts**, and evaluate artifact (for example check that an image is signed). Checks sit on the environment or service connection, not in YAML, so developers cannot remove them by editing the pipeline.

32 Explain service connection authentication options for Azure. Which do you recommend?

INTERMEDIATE | Security

- **Workload identity federation (OIDC):** recommended. No secret to store or rotate; Azure DevOps presents a short-lived token to Entra ID.
- **Service principal with secret/certificate:** older approach. Secrets expire and can leak, which breaks pipelines at 2 a.m.
- **Managed identity:** for self-hosted agents running on Azure VMs, the agent's identity is used.

Whichever you choose, scope the role to the minimum (resource group, not subscription) and restrict the connection to specific pipelines.

33 When would you use self-hosted agents and how do you scale and maintain them?

INTERMEDIATE | Agents

When you need private network access (databases, private AKS, on-prem), special software or hardware, longer caching, or lower cost at high volume. To scale: **Virtual Machine Scale Set agents** or **Managed DevOps Pools** (Microsoft-managed, scale-to-zero, can sit in your virtual network), or containerised ephemeral agents on Kubernetes. Prefer **ephemeral** agents (fresh per job) so there is no state leakage. Bake images with Packer, patch regularly, monitor queue time, and use `demands` to route jobs.

34 Explain the runOnce, rolling and canary strategies and their lifecycle hooks.

INTERMEDIATE | Deployments

runOnce executes the hooks once. **rolling** updates a number or percentage of VM targets at a time. **canary** exposes the new version to an increasing share (`increments`) before full rollout. Hooks: `preDeploy`, `deploy`, `routeTraffic`, `postRouteTraffic`, and `on: failure/success` where you place rollback or notification steps. For Kubernetes, canary needs a service mesh or ingress to split traffic, or a tool such as Argo Rollouts or Flagger.

canary.yml

```yaml
strategy:
  canary:
    increments: [10, 25]
    deploy:
      steps:
        - script: ./deploy.sh
    on:
      failure:
        steps:
          - script: ./rollback.sh
```

35 Compare blue-green, canary, rolling and recreate deployments.

INTERMEDIATE | Deployments

- **Recreate:** stop old, start new. Simple, but downtime.
- **Rolling:** replace instances gradually. No extra capacity, but old and new versions coexist and rollback is slower.
- **Blue-green:** two full environments, switch traffic at once. Instant rollback, but double the cost and tricky with database changes.
- **Canary:** send a small percentage to the new version, watch metrics, then expand. Lowest risk, needs good observability and traffic control.

Choose based on risk tolerance, cost, and whether versions can run side by side.

36 How do deployment slots work in Azure App Service?

INTERMEDIATE | Deployments

A slot is a live app instance with its own host name (for example `staging`) that shares the App Service plan. You deploy to staging, warm it up and test it, then **swap**: Azure exchanges the routing so production receives the warmed instance with no downtime. A bad release is rolled back by swapping again. Mark settings as **slot-specific** (sticky) so connection strings do not move with the swap, and use swap with preview for extra safety.

slot-swap.yml

```yaml
- task: AzureAppServiceManage@0
  inputs:
    azureSubscription: 'my-azure-connection'
    Action: 'Swap Slots'
    WebAppName: 'my-app'
    ResourceGroupName: 'my-rg'
    SourceSlot: 'staging'
```

37 Why use multi-stage Docker builds?

INTERMEDIATE | Containers

The first stage uses a full SDK image to compile; the final stage copies only the output into a small runtime image. Result: smaller images, faster pulls, a smaller attack surface (no compilers, no source), and builds that are self-contained and identical on any machine. Order layers so rarely-changing steps (restore dependencies) come before frequently-changing ones (copy source) to maximise layer caching.

38 How does AKS authenticate to ACR to pull images?

INTERMEDIATE | AKS

The cluster's kubelet managed identity is granted the **AcrPull** role on the registry, most simply by attaching the registry (`az aks update --attach-acr`). No image pull secrets are needed. If pods show `ImagePullBackOff`, check the role assignment, the image name and tag, that the registry is reachable (private endpoint and DNS), and that the tag actually exists.

39 Helm vs Kustomize vs plain manifests. What do you use?

INTERMEDIATE | AKS

Plain manifests: simple but you copy files per environment. **Kustomize:** overlays patch a base per environment, with no templating language, built into `kubectl`. **Helm:** templated, versioned packages with values files, releases and rollback (`helm rollback`), best for packaging apps and consuming third-party charts. I typically use Helm for apps with many settings, with `helm upgrade --install --atomic --wait` so a failed rollout rolls back automatically.

40 How do you run Terraform safely in a pipeline?

INTERMEDIATE | IaC

- **Remote state** in Azure Storage (with locking), never in the repo.
- Separate **plan** and **apply** stages: publish the plan file as an artifact and apply exactly that plan.
- **Approval** on the apply environment for production.
- Authenticate with workload identity, least privilege.
- Pin provider and Terraform versions, run `fmt`, `validate` and a policy or security scan (Checkov, tfsec).
- One state per environment or component, to limit blast radius.

terraform-pipeline.yml

```yaml
- stage: Plan
  jobs:
    - job: plan
      steps:
        - script: terraform init && terraform plan -out=tfplan
        - publish: tfplan
          artifact: plan
- stage: Apply
  dependsOn: Plan
  jobs:
    - deployment: apply
      environment: prod   # approval check lives on this environment
      strategy:
        runOnce:
          deploy:
            steps:
              - script: terraform apply tfplan
```

41 Bicep vs ARM templates vs Terraform. How do you choose?

INTERMEDIATE | IaC

Bicep is a cleaner language that compiles to ARM, with day-one support for new Azure features and no state file to manage. **ARM JSON** is the underlying format, verbose. **Terraform** is multi-cloud, has a large module ecosystem and a plan step, but needs state management. Choose Bicep for Azure-only teams that want simplicity, Terraform if you span clouds or already have the skills and modules. Use `what-if` (Bicep) or `plan` (Terraform) before every apply.

42 How do you integrate Azure Key Vault with pipelines?

INTERMEDIATE | Security

Three ways: a **variable group linked to Key Vault**, the **AzureKeyVault@2 task** which fetches chosen secrets as secret variables at runtime, or the **application reading Key Vault itself** through managed identity (best for runtime secrets, so the pipeline never touches them). Give the service connection only `get` and `list` on secrets via RBAC. Use short-lived references (Key Vault references in App Service) where possible.

key-vault.yml

```yaml
- task: AzureKeyVault@2
  inputs:
    azureSubscription: 'my-azure-connection'
    KeyVaultName: 'my-kv'
    SecretsFilter: 'DbPassword'
```

43 What security scanning would you add to a pipeline (DevSecOps)?

INTERMEDIATE | Security

- **Secret scanning** (push protection) so credentials never land in the repo.
- **SAST:** static analysis of source code (CodeQL, SonarQube).
- **SCA:** vulnerable and license-problem dependencies (Dependabot, Mend, Defender).
- **Container scanning** of the built image (Trivy, Defender for Containers).
- **IaC scanning** (Checkov, tfsec).
- **DAST:** scan the running test environment (OWASP ZAP).

Shift left: fast checks on every PR, heavier scans nightly. Fail the build only on agreed severity thresholds, or developers will start ignoring it.

44 What are quality gates and how do you implement them?

INTERMEDIATE | Quality

A quality gate is an automated pass/fail rule that blocks progress: tests pass, code coverage above a threshold (and on new code), no new critical vulnerabilities, SonarQube quality gate green, no linting errors. Implement them as build validation on the PR branch policy, and as checks on environments. The gate must be **objective and automated**, so it is consistent and nobody argues about it in a meeting.

45 What is your test strategy in a pipeline?

INTERMEDIATE | Quality

Follow the **test pyramid**: many fast unit tests on every commit, fewer integration tests (with real database or service containers), a small number of end-to-end/UI tests after deployment to a test environment, plus smoke tests after each deployment. Publish results with `PublishTestResults` and code coverage so trends are visible. Run the fast tests first so failures return in minutes, and run slow suites in parallel or nightly.

46 How do you deal with flaky tests?

INTERMEDIATE | Quality

First **detect and track** them (Azure DevOps flaky test detection, failure history). Then **quarantine** them so they do not block merges while someone fixes them, with an owner and deadline. Find root causes: timing, shared state, test order, external dependencies, hard-coded data. Avoid blanket retries as a permanent fix because they hide real problems, though a limited retry on end-to-end tests is acceptable with visibility.

47 A pipeline is slow. How do you speed it up?

INTERMEDIATE | Performance

Measure first: look at the timeline for the slowest steps. Then: cache dependencies and Docker layers, run independent jobs in parallel, shard tests, use shallow clone (`fetchDepth`), apply path filters, skip unnecessary work on docs-only changes, use a faster or larger agent (or self-hosted with warm caches), avoid rebuilding what has not changed, and move slow scans to nightly runs. Track pipeline duration and queue time as metrics.

48 Show an example of using conditions to deploy only from main.

INTERMEDIATE | Pipelines

Use a stage condition combining success with a branch check. This keeps pull request and feature-branch runs from deploying while still building and testing them.

condition.yml

```yaml
- stage: Deploy
  dependsOn: Build
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
```

49 How do you secure Azure DevOps pipelines themselves?

INTERMEDIATE | Security

- Least-privilege **build service identity**; limit job authorization scope to the current project.
- Restrict who can edit pipelines and who can use each service connection and variable group (pipeline permissions).
- Use **protected resources** with approvals and checks.
- Do not expose secrets to PRs from forks.
- Pin task and template versions; review marketplace extensions.
- Use **required templates** so mandatory security steps cannot be removed.
- Audit through the Auditing log, streamed to Log Analytics.

50 What are required templates, extends and pipeline decorators?

INTERMEDIATE | Governance

With `extends`, a pipeline must be built from a central template that defines the allowed structure, so teams can only plug in their own steps. A **required template check** on a protected resource rejects any pipeline that does not extend the approved template. **Pipeline decorators** are org-level extensions that inject steps (for example a compliance scan) into every job automatically. Together they enforce standards without relying on each team to remember.

extends.yml

```yaml
extends:
  template: templates/secure-pipeline.yml@platform
  parameters:
    buildSteps:
      - script: npm ci
      - script: npm test
```

51 How do you build from multiple repositories in one pipeline?

INTERMEDIATE | Pipelines

Declare them under `resources: repositories` (type `git` for Azure Repos or `github`), then use `checkout: self` and `checkout: repoAlias` steps. Multiple checkouts land in separate folders under `$(Build.SourcesDirectory)`. Typical uses are shared templates in a platform repo, shared libraries, or infrastructure code. Be careful that the build service identity has read access to the other repository.

52 How do you handle a monorepo in Azure Pipelines?

INTERMEDIATE | Pipelines

Use **path filters** in `trigger` and `pr` so each service pipeline only runs when its folder (or shared code) changes. Keep one pipeline per service using shared templates, and use a conditional `paths` filter for shared libraries. For finer control, detect changed paths in a first job and set outputs that condition later jobs. Use caching and a shallow checkout to keep builds fast.

path-filters.yml

```yaml
trigger:
  branches:
    include: [main]
  paths:
    include:
      - services/orders/*
      - shared/*
```

53 How do retention policies work for pipelines?

INTERMEDIATE | Artifacts

Runs and artifacts are deleted after the retention period (the default is 30 days for runs, set at project level). Runs used by a release or deployment, or **retained by a lease**, are kept longer. Set artifact and run retention deliberately: long enough for compliance and rollback, short enough to control storage cost. For published packages, use Azure Artifacts retention rules and keep release versions permanently.

54 How do you automate Azure DevOps itself through the REST API or CLI?

INTERMEDIATE | Automation

The `az devops` CLI extension and REST API can create pipelines, queue runs, manage variable groups, service connections, permissions and branch policies. I use them to bootstrap new projects consistently, to run pipelines from scripts, and to report on usage. Authenticate with a PAT (scoped, short-lived) or better a service principal or managed identity. The same ideas can be expressed in Terraform with the Azure DevOps provider for repeatable project setup.

55 What is your rollback strategy?

INTERMEDIATE | Deployments

Roll forward when the fix is small and fast; **roll back** when impact is high. Make rollback cheap by design: immutable artifacts (redeploy the previous version), slot swap back, `helm rollback`, blue-green traffic switch, or feature flag off. Keep the previous artifacts available, make database changes backward compatible so the old code still works, and rehearse rollback. Define the trigger up front (error rate, failed health checks) and automate it where possible.

56 How do you handle database changes in CI/CD?

INTERMEDIATE | Databases

Treat schema as code: versioned migration scripts (EF Core migrations, Flyway, Liquibase, DACPAC) stored in the repo and applied by the pipeline in the same order across environments. Make changes **backward compatible** with the expand-and-contract pattern: add the new column, deploy code that uses both, migrate data, then remove the old column in a later release. This allows zero-downtime deploys and safe rollback. Always back up or snapshot before production migrations.

57 Feature flags vs feature branches. When do you use flags?

INTERMEDIATE | Practices

Long-lived feature branches delay integration and cause painful merges. **Feature flags** let you merge unfinished code to `main` safely, deploy it switched off, and release it by configuration, so deployment and release are decoupled. Flags also support gradual rollouts, A/B tests and an instant kill switch. The cost is flag hygiene: remove old flags, test both paths, and keep ownership clear. Azure App Configuration or LaunchDarkly are common providers.

58 What is GitOps and how does it differ from push-based pipeline deployments?

INTERMEDIATE | GitOps

In **push-based** CD, the pipeline holds cluster credentials and applies changes. In **GitOps** the desired state lives in Git and an in-cluster agent (Flux or Argo CD) **pulls** and reconciles it continuously. Benefits: no cluster credentials in the CI system, drift is auto-corrected, Git history is the audit log, and rollback is a `git revert`. The pipeline's job becomes building the image and updating the manifest repo. AKS has a Flux extension (GitOps) built in.

59 How do you close the feedback loop after deployment? Which metrics matter?

INTERMEDIATE | Operations

Use Application Insights/Azure Monitor with deployment annotations, dashboards and alerts, and run smoke tests after every release. Track the four **DORA metrics**: deployment frequency, lead time for changes, change failure rate and time to restore service. Add pipeline metrics (duration, queue time, flakiness). Feed incidents back into the pipeline as new tests or checks, so each failure improves the process.

60 How do you manage environment-specific configuration across dev, test and prod?

INTERMEDIATE | Configuration

Keep **one artifact** and inject configuration at deployment: per-environment variable templates or variable groups, Azure App Configuration, and Key Vault for secrets. Never rebuild the app per environment. Structure the pipeline so each environment is a stage reusing the same stage template with different parameters. Keep the differences small and visible, and validate configuration at startup so mistakes fail fast.

env-variables.yml

```yaml
variables:
  - template: vars/${{ parameters.env }}.yml
```

61 What are release gates / automated checks and how would you use them?

INTERMEDIATE | Deployments

Automated checks that run before or after a deployment and decide whether to proceed: query Azure Monitor alerts (no active alerts), call an Azure Function or REST API (is the change window open, is a ticket approved), run a smoke test, or check an artifact. After deployment they can hold a stage while a **soak period** monitors health, and fail the run if metrics regress. Implemented as environment checks in YAML pipelines (the old "gates" in Classic).

62 What image tagging strategy do you recommend?

INTERMEDIATE | Containers

Tag every image with something **immutable and traceable**: commit SHA or build ID (optionally plus semantic version). Avoid deploying `latest` because it is mutable and breaks rollback and reproducibility. For the strictest guarantee, deploy by **digest** (`image@sha256:...`). Add labels (source commit, build URL), keep ACR retention policies to remove old untagged images, and lock production tags against overwrite.

63 What are agentless (server) jobs and when do you use them?

INTERMEDIATE | Pipelines

A server job (`pool: server`) runs in Azure DevOps itself without an agent. It supports a small set of tasks: **ManualValidation**, **Delay**, **InvokeRESTAPI** and **AzureFunction**. Use it for manual approvals inside a stage, waiting for an external process, or calling a webhook, without holding an agent (and its cost) while waiting.

server-job.yml

```yaml
- job: waitForApproval
  pool: server
  steps:
    - task: ManualValidation@0
      timeoutInMinutes: 1440
      inputs:
        notifyUsers: 'lead@contoso.com'
        instructions: 'Approve prod rollout'
        onTimeout: reject
```

64 How do you debug a failing pipeline?

INTERMEDIATE | Troubleshooting

Read the failing step's log first, expanding the error. Re-run with **system diagnostics** (`system.debug: true`) for verbose logs. Compare with the last green run: what changed in code, template, task version, agent image, or secrets? Reproduce locally or with a debug step on a self-hosted agent. Check the usual suspects: expired credentials, permission changes, agent image updates, network and DNS, unpinned versions, and cache problems. Fix the cause, then add a check so it is caught earlier next time.

65 How do you manage dependencies and packages with Azure Artifacts?

INTERMEDIATE | Artifacts

Create a feed per team or organisation with **upstream sources** (nuget.org, npmjs, Maven Central) so builds pull public packages through your feed, which caches them and protects against upstream outages or removals. Publish internal libraries as versioned packages and consume them through the feed. Control access by feed view (`@Local`, `@Prerelease`, `@Release`) to promote packages, and add `nuget.config` or `.npmrc` to point builds at the feed.

C Scenario questions (66-100)

Design, troubleshooting and judgment

66 Design a CI/CD pipeline for a microservices application on AKS from scratch.

SCENARIO | Design

Clarify scale and compliance first, then describe the flow.

- **CI (per service):** PR validation (build, unit tests, lint, SAST, secret scan), path filters, shared stage templates.
- **Build once:** Docker multi-stage build, scan the image, push to ACR tagged with commit SHA, optionally sign it.
- **CD:** a Helm chart or Kustomize overlay per environment. Deploy dev automatically, then test and prod through environments with approvals and checks.
- **Verification:** smoke tests, health checks, canary or rolling with automatic rollback.
- **Platform:** workload identity for auth, Key Vault for secrets, IaC (Terraform/Bicep) for the cluster, and monitoring with DORA metrics.

Mention GitOps as an alternative to push deployments.

67 Your pipeline takes 45 minutes and developers complain. What do you do?

SCENARIO | Performance

1. **Measure:** read the timeline and find the longest steps and queue time.
2. **Quick wins:** cache dependencies and Docker layers, shallow clone, parallelise independent jobs, shard tests.
3. **Reduce work:** path filters, skip heavy scans on every PR (run nightly), build only changed services.
4. **Resources:** faster agents, warm self-hosted agents, more parallel jobs if queueing.
5. **Prevent regression:** track duration as a metric with a target.

Report improvements to the team so they see the value.

68 A production deployment fails halfway. What do you do?

SCENARIO | Incident

Contain first: stop the rollout, and if users are affected, roll back to the last good version (swap slots, `helm rollback`, redeploy the previous artifact) rather than debugging live. Confirm service health. Then **diagnose**: pipeline logs, deployment events, application logs and recent changes. Was it code, config, infrastructure or a dependency? Fix forward in lower environments, redeploy through the normal path. Finally run a **blameless post-incident review** and add the missing test, check or gate. If a migration ran halfway, assess data state before any rollback.

69 How would you achieve zero-downtime deployment when a database schema changes?

SCENARIO | Design

Use the **expand-and-contract** pattern. Release 1: add the new column or table in a backward-compatible way (nullable or default). Release 2: deploy code that writes to both and reads the new one, and backfill data. Release 3: remove the old column once nothing uses it. Combine with a deployment strategy that allows old and new code to coexist (rolling, canary, blue-green or slot swap), and make migrations idempotent. Never ship a breaking schema change in the same release as the code that needs it.

70 A developer committed a secret to the repo, or a secret appeared in a pipeline log. What now?

SCENARIO | Security

Treat it as compromised immediately: rotate/revoke the secret first, before cleaning anything up. Then check access logs for misuse. Remove it from history (`git filter-repo`) and logs if needed, though rotation is what actually protects you. Prevent recurrence: enable secret scanning with push protection, use Key Vault and managed identity so secrets are not handled by developers, and mask values. Share the lesson without blame.

71 You must migrate 150 Classic pipelines to YAML. How do you plan it?

SCENARIO | Migration

Inventory and classify pipelines (active, stale, duplicated). Delete what is unused. Group by pattern (for example .NET web apps, Node apps, container apps) and build **shared templates** per pattern. Migrate in waves, starting with low-risk pipelines, running old and new in parallel and comparing results. Use the build pipeline's "export to YAML" as a start, but rebuild release pipelines as multi-stage with environments, approvals and variable groups. Train teams, document, then disable Classic and set a decommission date.

72 You are asked to standardise pipelines for 50 teams without slowing them down. How?

SCENARIO | Governance

Provide a **paved road**: opinionated, well-documented templates in a central repo that cover build, test, scan and deploy, so teams get a working pipeline in minutes. Enforce the non-negotiables (security scans, approvals) with `extends`, required templates and decorators, but leave extension points (extra steps, parameters). Version templates with tags and communicate changes. Offer support and gather feedback. If the standard is easier than rolling your own, adoption follows without force.

73 Jobs are queueing for 20 minutes on your self-hosted pool. How do you fix it?

SCENARIO | Agents

Check capacity versus demand: queue time, agent utilisation, peak hours. Short term: add agents or raise the scale set maximum. Long term: move to **autoscaling ephemeral agents** (scale set agents, Managed DevOps Pools or Kubernetes-based agents), split pools by workload so heavy jobs do not starve light ones, remove idle waiting steps, reduce job duration with caching, and confirm that parallel job licences are not the real limit. Add an alert on queue time.

74 The pipeline works in dev but fails in prod. How do you troubleshoot?

SCENARIO | Troubleshooting

Find the **differences** between environments: configuration and variables, secrets, permissions and RBAC, network rules and private endpoints, different agent or pool, data volume, resource SKUs, and artifact versions (confirm the same artifact is deployed). Read the failing step's logs and the application logs. Test the same step manually on a prod-like agent. Fix the cause, and reduce drift by defining environments in IaC and parameterising them from the same templates.

75 Design multi-environment promotion with approvals, using the same artifact everywhere.

SCENARIO | Design

Build once in the Build stage and publish the artifact (or push the image with an immutable tag). Then define stages **Dev, Test, Prod** that each use the same deploy template with different parameters. Dev deploys automatically; Test runs integration tests; Prod requires **approval and checks** on the environment (branch control, business hours, monitor alerts). Because the artifact is identical, what you tested is exactly what reaches production. Configuration is injected per environment, never baked into the build.

76 Developers are bypassing the pipeline and deploying to production manually. How do you stop it?

SCENARIO | Governance

Remove the ability, not just the habit: give developers **read-only** access to production resources, and only the pipeline's service connection (workload identity) has write rights. Restrict that connection to the production pipeline and add approvals and branch control. Use Azure Policy to deny or alert on out-of-band changes, and alert on activity-log writes that do not come from the pipeline identity. Offer a break-glass process with audit, and make the pipeline fast enough that bypassing is not tempting.

77 Your pipelines fail intermittently with no code change. What is your approach?

SCENARIO | Troubleshooting

Collect evidence: failure rate, which steps, which agents, which times. Common causes: flaky tests, network or registry timeouts, rate limits (Docker Hub, package feeds), unpinned dependencies or task versions, agent image updates, shared state on self-hosted agents, and resource exhaustion. Mitigate: pin versions, mirror dependencies in Azure Artifacts and ACR, add limited retries (`retryCountOnTaskFailure`) for network steps, use ephemeral agents, and quarantine flaky tests. Track the failure rate so you can prove improvement.

78 How would you secure the software supply chain in your pipelines?

SCENARIO | Security

- **Pin and verify** dependencies, task versions and base images (by digest); use a private feed with upstream sources.
- **Scan** dependencies and images for vulnerabilities and licences.
- Generate an **SBOM** and store it with the artifact.
- **Sign** images and artifacts (Notation or Cosign) and verify signatures at deploy time with an admission policy or an environment check.
- Use least-privilege, short-lived credentials (OIDC), protected branches and required reviews.
- Harden agents and keep build definitions under review.

79 How do you deploy the same application to multiple Azure regions?

SCENARIO | Design

Use a stage per region (or a matrix of regions) with the same artifact and template, parameterised by region. Roll out **region by region** (a pilot region first, then the rest) with checks between waves, rather than everything at once. Front with Azure Front Door or Traffic Manager for routing and failover. Keep infrastructure identical through IaC modules, replicate the registry (ACR geo-replication), and handle data and secrets per region. Monitor each region separately.

80 A critical bug is in production while the release branch is frozen. How do you handle the hotfix?

SCENARIO | Process

Branch the hotfix from the **production tag/commit** (not from `main`, which may contain unreleased work). Fix it with a test, open a PR with expedited review, and run the full pipeline (do not skip tests, but use a lean path if available). Deploy through the normal pipeline with approvals, using a fast-track approver. Then **merge the fix back** to `main` and any open release branch so it is not lost. Afterwards, review why it escaped and add a regression test.

81 A service principal secret expired and every pipeline broke on a Monday morning. How do you prevent a repeat?

SCENARIO | Security

Immediate fix: rotate the secret and update the service connection. Permanent fix: migrate connections to **workload identity federation**, which has nothing to expire, or managed identity on agents. Where secrets remain, store them in Key Vault with expiry alerts (Event Grid, Azure Monitor), and keep an inventory of owners and expiry dates. Document the runbook and test service connections regularly.

82 You have a monorepo with 30 services. How do you avoid building everything on each commit?

SCENARIO | Pipelines

Use **path filters** on `trigger` and `pr` per service pipeline, with shared libraries included in the filters of the services that depend on them. For shared build logic use templates. For finer detection, run a first job that diffs against the target branch and emits outputs that condition later jobs. Add caching, and consider build tools with affected-project detection (Nx, Bazel, Turborepo). Always run the full build on a schedule to catch gaps in the filters.

83 A Terraform pipeline fails with a state lock error, and another reports drift. What do you do?

SCENARIO | IaC

State lock: first check whether another run is actually in progress. If not (crashed run), confirm and release with `terraform force-unlock <ID>` (for Azure Storage backends the lock is a blob lease). Never force-unlock during an active apply. **Drift:** run `terraform plan` to see what changed outside Terraform, decide whether to revert the manual change or import and codify it. Prevent: lock down portal access, run scheduled drift-detection plans, and use one pipeline per state with an exclusive lock.

84 The pipeline reports success but pods are in CrashLoopBackOff or ImagePullBackOff. What now?

SCENARIO | AKS

A "successful" apply only means Kubernetes accepted the manifest. Use `kubectl describe pod` and `kubectl logs --previous`. **ImagePullBackOff:** wrong image name or tag, registry permission (AcrPull), or network/DNS. **CrashLoopBackOff:** application error, missing config or secret, bad environment variables, failing probes or insufficient memory. Fix by rolling back (`kubectl rollout undo`/`helm rollback`). Improve the pipeline: use `rollout status` or Helm `--atomic --wait` so failed rollouts fail the pipeline, add readiness probes and a post-deploy smoke test.

85 You are moving a company from Jenkins to Azure DevOps. How do you approach it?

SCENARIO | Migration

Inventory jobs and plugins, and find out what each plugin is really doing. Map concepts (Jenkinsfile stages to YAML stages, shared libraries to templates, credentials to service connections and Key Vault, agents to pools). Standardise first, then migrate **in waves**, pilot team first, running both systems in parallel until results match. Reimplement integration points (SCM, artifact storage, notifications, secrets). Train people, document the new standards, and set a Jenkins shutdown date so the migration actually finishes.

86 How would you reduce the cost of build infrastructure?

SCENARIO | Cost

Measure first (minutes per pipeline, idle agent time, parallel job usage). Then: **ephemeral autoscaling agents that scale to zero**, right-size VM SKUs, use spot instances for non-critical builds, cache dependencies, skip unnecessary runs with path filters and PR-only triggers, cancel superseded runs, shorten retention for artifacts and logs, clean ACR images, and delete stale pipelines. Make cost visible per team with tags and reports.

87 An auditor asks for evidence that production changes are approved and segregated. What do you show?

SCENARIO | Compliance

Show the full chain: work item to PR (with reviewers and branch policy) to pipeline run to **environment approval** (who approved, when) to deployment record. Enforce **segregation of duties**: the author cannot approve their own PR or deployment (set "requester cannot approve"), only the pipeline identity can write to production, and pipeline definitions are protected by required templates. Stream the **Auditing log** to Log Analytics for retention, and keep run logs and artifacts per the retention policy.

88 Implement a canary release with automatic rollback if the error rate rises.

SCENARIO | Deployments

Deploy the new version to a small share of traffic (10%) using a traffic splitter: ingress weights, a service mesh, Front Door, or **Argo Rollouts/Flagger** on AKS. Define the analysis: error rate, latency and saturation from Azure Monitor or Prometheus. Hold at each step for a soak period, and promote only if metrics stay within thresholds; otherwise abort and route all traffic back to the stable version automatically. In Azure Pipelines use the `canary` strategy with an `on: failure` hook, plus an Azure Monitor check. Alert the team on every rollback.

89 The main branch keeps breaking. How do you keep it green?

SCENARIO | Process

Make it **impossible to merge broken code**: branch policies requiring PR build validation, passing tests, and up-to-date branches (or a merge queue so the merge result is tested). Keep PR builds fast so people do not avoid them, fix flaky tests, and make the rule that a broken `main` is fixed or reverted immediately, before any new work. Encourage small PRs and trunk-based development. Track broken-build time as a team metric.

90 Explain "build once, deploy many". Why does it matter?

SCENARIO | Design

You build the artifact or image a single time, then promote **that same artifact** through every environment, changing only configuration. This guarantees that what you tested is what runs in production; rebuilding per environment can introduce differences (new dependency versions, different tool versions). It also speeds up promotion, supports rollback to a known artifact, and gives clean traceability from a production version back to one commit and one build.

91 A production approval has been pending for three days and the change window is closing. How do you handle it?

SCENARIO | Process

Check why: wrong approver, missing notification, or unclear risk. Escalate to the backup approver (use **groups**, not single people, as approvers). Configure **timeouts** so a pending approval expires and rejects rather than hanging, and reminders to approvers. Limit deployments to business hours with a check, and keep a documented emergency path with extra audit. For the long term, make changes smaller and lower-risk so more can flow through automated checks.

92 Service B must build only after Service A publishes a new version. How do you wire that up?

SCENARIO | Pipelines

Use a **pipeline resource trigger**: B declares A as `resources: pipelines` with `trigger`, so B starts when A completes (with branch and tag filters). Better still, depend on **published versions**: A publishes a package or image to a feed or registry, and B consumes a pinned version, updated by a bot or a PR (for example Dependabot). That avoids tight coupling and lets each team release independently. Use contract tests to catch breaking changes.

93 Part of the workload runs on on-premises servers. How do you deploy to it from Azure Pipelines?

SCENARIO | Hybrid

Register the servers as **Virtual machine resources** in an Environment (installing the environment agent), and use a `deployment` job with the `rolling` strategy to update them in batches with health checks. Alternatively run a **self-hosted agent** inside the network that can reach the servers. Use Key Vault or an on-prem vault for secrets, keep outbound-only connectivity from agent to Azure DevOps for security, and keep artifacts identical to those used in the cloud.

94 How do you publish a shared library to teams with proper versioning?

SCENARIO | Versioning

Use semantic versioning driven by the pipeline (GitVersion or a version file). On `main`, the pipeline builds, tests and publishes a stable version to the Azure Artifacts feed; pre-release versions (`-beta.N`) come from branches or PRs, visible through feed views. Consumers pin versions, and **breaking changes require a major bump** with release notes. Never overwrite a published version; packages are immutable. Automatically open update PRs in consumer repos, and keep a changelog.

95 A team complains that the pipeline is too restrictive and slows them down. How do you respond?

SCENARIO | Culture

Listen and measure: which check, how much time, how often does it fail legitimately? Separate **valuable controls** from **friction**. Speed up what is slow (caching, parallelism, fast PR checks), fix flaky gates, and move heavy checks off the critical path where risk allows. Keep security and quality gates non-negotiable but explain the reason behind each. Offer an exception process with audit instead of silent workarounds. The goal is to make the safe way the easy way.

96 How would you protect against loss of Azure DevOps data, such as a repo being deleted?

SCENARIO | Resilience

Azure DevOps is a managed service, but you are responsible for your content. Enable **soft delete and recovery** for repositories, restrict who can delete projects or repos, and keep **regular backups or mirrors** (scheduled clone of repos to another location, export of work items, pipeline definitions as YAML in Git). Keep service connections and variable group definitions reproducible through IaC. Document and test the recovery procedure.

97 With 200 microservices, ACR is full of old images. How do you manage this?

SCENARIO | Containers

Define a **retention policy**: purge untagged manifests (ACR retention policy), and use scheduled `acr purge` tasks to delete old tags by age or pattern while keeping a number of recent versions and everything currently deployed to production. Protect release tags with locks. Use tag conventions that make rules simple (for example PR or branch builds expire quickly, release tags are kept). Consider geo-replication and registry tiers only where needed, and monitor storage so cost is visible.

98 You hit parallel job limits and builds are queueing across the organisation. What are your options?

SCENARIO | Agents

Look at usage per project to see who consumes the most, then decide. Options: buy more **Microsoft-hosted parallel jobs**, add **self-hosted parallelism** (self-hosted agents are billed differently) through scale sets or Managed DevOps Pools, reduce unnecessary runs (path filters, cancel superseded builds, PR-only triggers), shorten jobs, and set priorities by pool or time. Add a dashboard of queue time so you can justify the investment with data.

99 You must roll out a new mandatory security scan to every pipeline with minimal disruption. How?

SCENARIO | Governance

Start with a **pilot** on a few volunteer teams and tune the scan (noise, speed, thresholds). Add it to the shared templates and, for teams on custom pipelines, a **pipeline decorator** or required template. Begin in **report-only** mode so teams see results without being blocked, publish a timeline, then move to blocking for critical findings and later for high. Provide documentation, an exception process and support. Communicate early, and track adoption and fix rates.

100 Tell me about a difficult CI/CD problem you solved. (Behavioural question)

SCENARIO | Behavioral

Use **STAR**: Situation, Task, Action, Result. Choose a real story with numbers. Example shape: *"Our release pipeline took 50 minutes and failed about 20% of runs (situation). I was asked to make releases reliable (task). I measured step times, added caching and parallel test jobs, moved to ephemeral agents, quarantined flaky tests, and pinned dependency versions (action). Duration fell to 14 minutes, failures to under 3%, and we went from weekly to daily deployments (result)."* Finish with what you learned. Prepare two or three such stories before the interview: one performance, one incident, one improvement of process or security.

Questions you should ask the interviewer

At 5+ years, good questions show seniority. Pick two or three:

- How often do you deploy to production today, and what is the change failure rate?
- Are the pipelines YAML and template-based, or is there Classic legacy to migrate?
- How are agents managed: Microsoft-hosted, scale sets, or something else?
- How are secrets and service connections managed and rotated?
- What is the approach to environments, approvals and rollbacks for production?
- What does the on-call and incident process look like for deployment problems?
- Where do you want the delivery process to be in a year?

Final checklist before the interview

Done?	Item
[ ]	Can explain CI vs delivery vs deployment in under 30 seconds.
[ ]	Can draw the YAML hierarchy and write a basic pipeline from memory.
[ ]	Know the three variable syntaxes and when each is evaluated.
[ ]	Can explain workload identity federation and why it beats secrets.
[ ]	Can compare blue-green, canary, rolling and recreate.
[ ]	Can describe a Terraform pipeline with plan, approval, apply.
[ ]	Can walk through an AKS deployment and debug ImagePullBackOff.
[ ]	Have 3 STAR stories with numbers (performance, incident, improvement).
[ ]	Have 2 or 3 questions to ask the interviewer.

Good luck
Practise answering aloud, not just reading. Time yourself: basic answers in 30 seconds, intermediate in about a minute, scenarios in two to three minutes with the clarify, contain, diagnose, fix, prevent structure.